

## Cloud-Storage-Assignment-4

**Q1. Create a table comparing Amazon EBS and Amazon S3, listing at least 4 key differences (such as data type, access method, use case, and scalability) and give one real-world app example where each would be the better choice.**

**Ans:**

| Feature       | Amazon EBS                                                                                | Amazon S3                                                                          |
|---------------|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Full Form     | Elastic Block Store                                                                       | Simple Storage Service                                                             |
| Data Type     | Block storage; mainly files, OS, databases and application data                           | Object storage; stores files/objects such as images, videos, documents and backups |
| Access Method | Attached to an EC2 instance and accessed like a disk/drive                                | Accessed through APIs, AWS Console, CLI or applications over the network           |
| Main Use Case | Operating systems, databases and applications running on EC2                              | Backup, media files, static website files, logs and large data storage             |
| Scalability   | Volume size can be increased, but storage is tied to EBS volumes/availability-zone design | Highly scalable object storage designed to store very large amounts of data        |
| Performance   | Provides low-latency block storage suitable for                                           | Optimized for object storage and large-                                            |

| Feature                   | Amazon EBS                                                                                                                                   | Amazon S3                                                                                                                                |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
|                           | applications requiring frequent read/write operations                                                                                        | scale data access rather than disk-like block operations                                                                                 |
| <b>Real-World Example</b> | <b>Online banking application:</b> EBS can store the database running on an EC2 server because the application needs fast read/write access. | <b>Instagram-like application:</b> S3 can store users' uploaded photos and videos because it is suitable for large-scale object storage. |

### Conclusion:

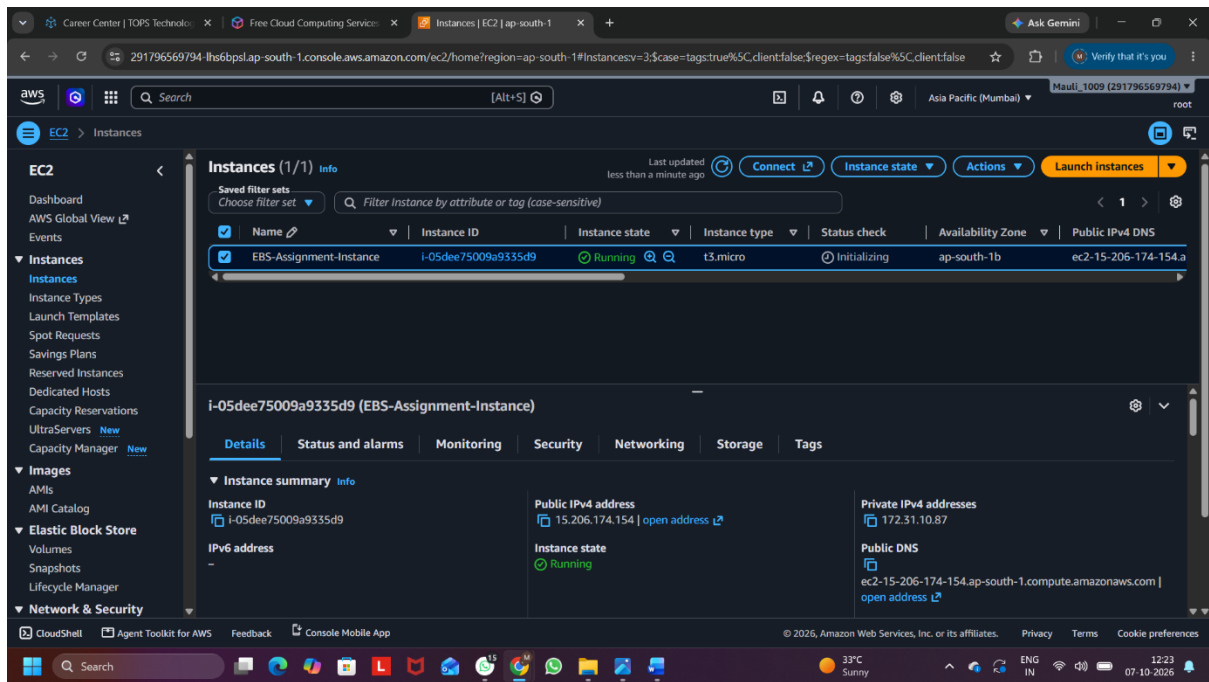
Amazon EBS is better suited for **EC2-based applications that require fast, disk-like storage**, while Amazon S3 is better suited for **storing and retrieving large amounts of files and objects at scale**.

**Q2. Launch a new EC2 instance in the AWS Free Tier and attach a new EBS volume of at least 2 GB to it using the AWS Management Console. Hint: After launching the instance, use the 'Volumes' section in EC2 to create and attach the volume.**

**Ans:**

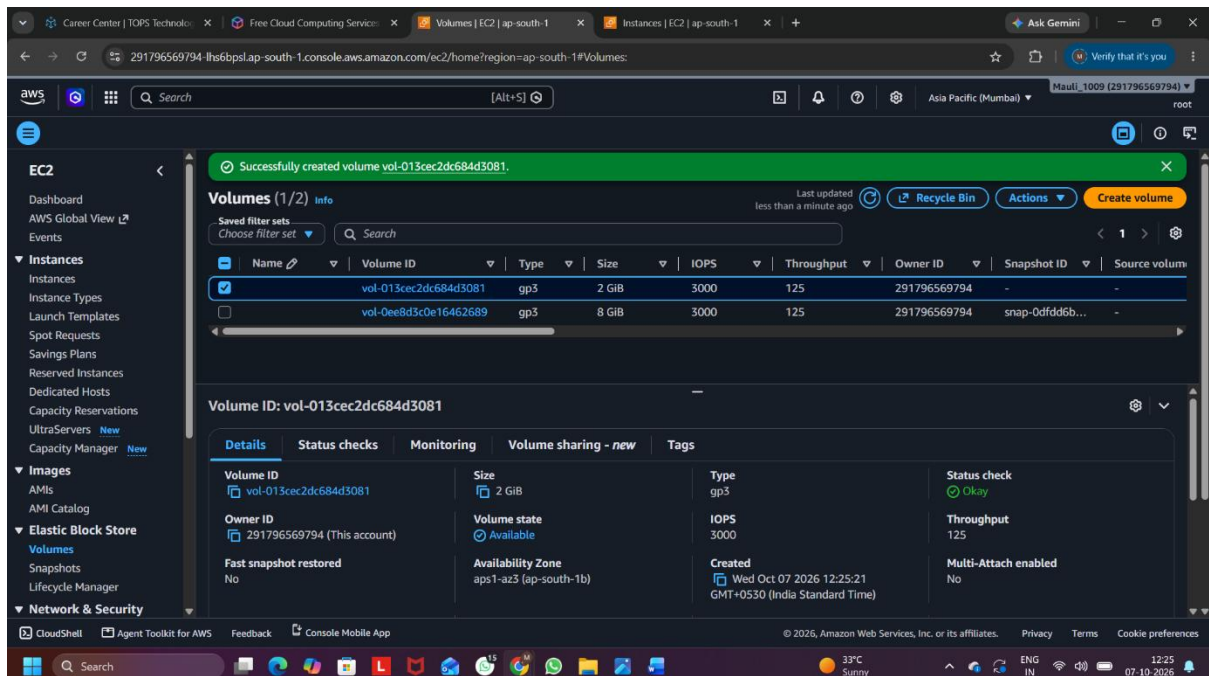
### Procedure

**Step 1:** Open AWS Management Console → EC2 → Instances → **Launch instances**. Select a **Free Tier eligible** AMI and instance type, then click **Launch instance**.



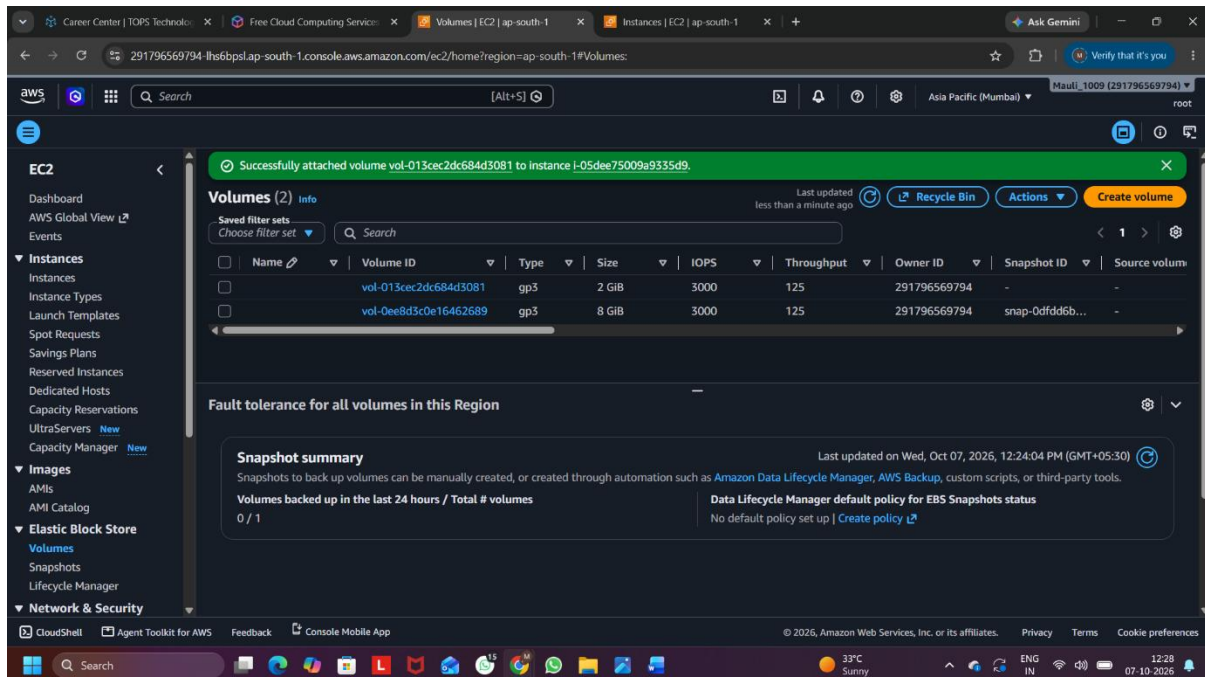
**Screenshot 1: EC2 instance successfully launched.**

**Step 2: Go to EC2 → Volumes → Create volume. Select gp3, set Size = 2 GiB, select the same Availability Zone as the EC2 instance, and click Create volume.**



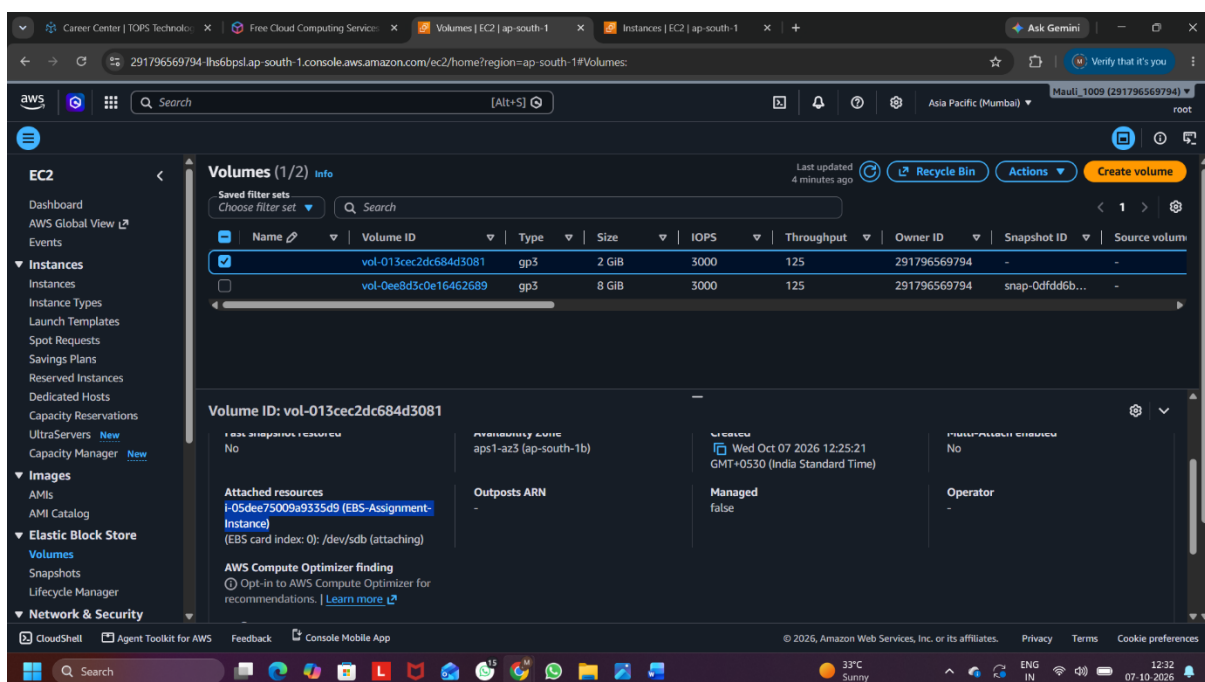
**Screenshot 2: 2 GiB EBS volume created successfully.**

**Step 3:** Select the newly created volume and choose **Actions** → **Attach volume**. Select the EC2 instance and choose **/dev/sdb** as the device name. Click **Attach volume**.



**Screenshot 3:** EBS volume attached to the EC2 instance.

**Step 4:** Verify that the EBS volume status is **In-use** and that it is attached to the selected EC2 instance.



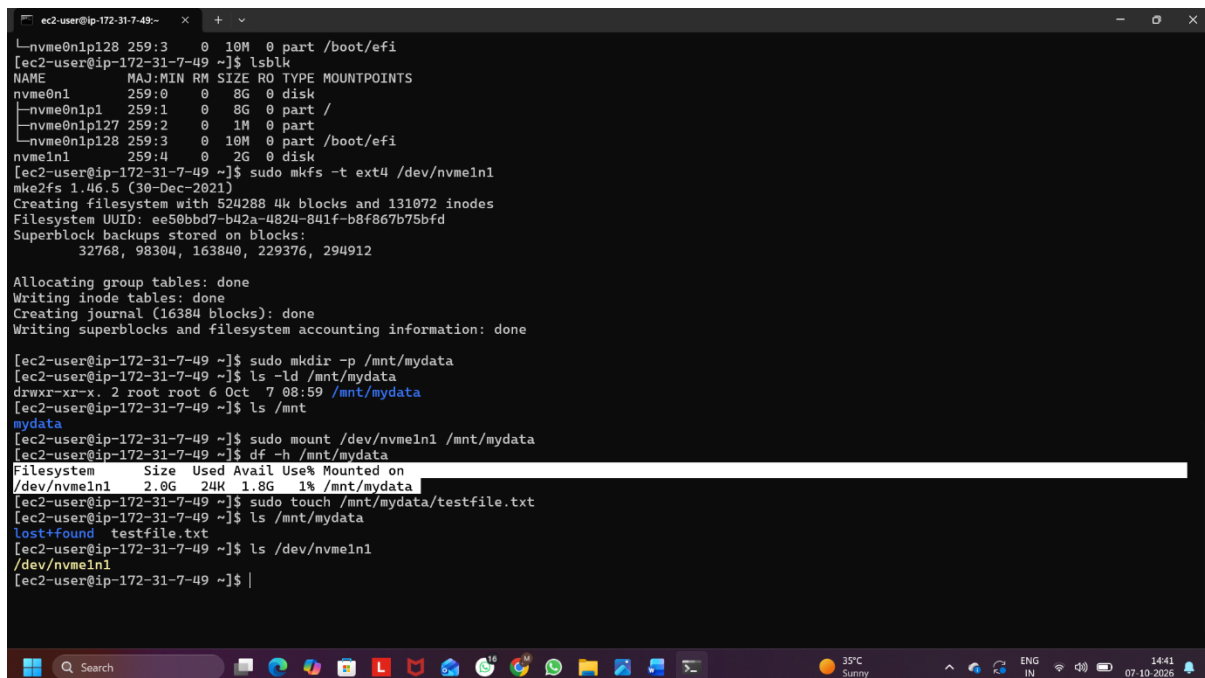
**Screenshot 4:** EBS volume successfully attached and in use.

## Result

The EC2 instance was successfully launched and a **2 GiB EBS volume** was created and attached to it.

**Q3.** Log in to your EC2 instance using SSH and format the attached EBS volume, then mount it to a new directory name `/mnt/mydata`. Hint: Use `lsblk` to find your volume, `mkfs` to format, and mount to attach it to the directory.

**Ans:**



```
ec2-user@ip-172-31-7-49:~$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
nvme0n1p128 259:3 0 10M 0 part /boot/efi
nvme0n1 259:0 0 8G 0 disk
├─nvme0n1p1 259:1 0 8G 0 part /
├─nvme0n1p127 259:2 0 1M 0 part
├─nvme0n1p128 259:3 0 10M 0 part /boot/efi
└─nvme1n1 259:4 0 2G 0 disk
[ec2-user@ip-172-31-7-49 ~]$ sudo mkfs -t ext4 /dev/nvme1n1
mke2fs 1.46.5 (30-Dec-2021)
Creating filesystem with 524288 4k blocks and 131072 inodes
Filesystem UUID: ee50bbd7-b42a-4824-841f-b8f867b75bfd
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

[ec2-user@ip-172-31-7-49 ~]$ sudo mkdir -p /mnt/mydata
[ec2-user@ip-172-31-7-49 ~]$ ls -ld /mnt/mydata
drwxr-xr-x. 2 root root 6 Oct 7 08:59 /mnt/mydata
[ec2-user@ip-172-31-7-49 ~]$ ls /mnt
mydata
[ec2-user@ip-172-31-7-49 ~]$ sudo mount /dev/nvme1n1 /mnt/mydata
[ec2-user@ip-172-31-7-49 ~]$ df -h /mnt/mydata
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme1n1    2.0G  24K  1.8G   1% /mnt/mydata
[ec2-user@ip-172-31-7-49 ~]$ sudo touch /mnt/mydata/testfile.txt
[ec2-user@ip-172-31-7-49 ~]$ ls /mnt/mydata
lost+found testfile.txt
[ec2-user@ip-172-31-7-49 ~]$ ls /dev/nvme1n1
[ec2-user@ip-172-31-7-49 ~]$
```

**Screenshot 1:** A test file is successfully created on the mounted EBS volume.

## 1. Connect to the EC2 instance using SSH.

`ssh -i "My-Web-Server-Key.pem" ec2-user@ec2-16-4-41-75.ap-south-1.compute.amazonaws.com`

## 2. Identify the attached EBS volume using `lsblk`.

`lsblk`

The attached **2 GB EBS volume** was identified as `/dev/nvme1n1`.

**3. Format the EBS volume with the ext4 filesystem.**

```
sudo mkfs -t ext4 /dev/nvme1n1
```

**4. Create the mount directory.**

```
sudo mkdir -p /mnt/mydata
```

**5. Mount the EBS volume to /mnt/mydata.**

```
sudo mount /dev/nvme1n1 /mnt/mydata
```

**6. Verify that the volume is mounted successfully.**

```
df -h /mnt/mydata
```

The output showed /dev/nvme1n1 mounted at /mnt/mydata.

**7. Verify the mounted storage by creating a test file.**

```
sudo touch /mnt/mydata/testfile.txt
```

```
ls -l /mnt/mydata
```

The testfile.txt file was successfully created inside the mounted EBS volume.

**Result**

The attached **2 GB EBS volume** was successfully formatted with **ext4** and mounted at **/mnt/mydata**. The successful creation of a test file confirmed that the mounted volume is accessible and working correctly.

**Q4. Simulate a Zomato-style image upload: create a text file named zomato\_menu.txt on your EC2 instance, copy it to your mounted EBS volume, and verify it is accessible from the new mount point.**

**Ans:**

```
ec2-user@ip-172-31-7-49:~  
Writing inode tables: done  
Creating journal (16384 blocks): done  
Writing superblocks and filesystem accounting information: done  
  
[ec2-user@ip-172-31-7-49 ~]$ sudo mkdir -p /mnt/mydata  
[ec2-user@ip-172-31-7-49 ~]$ ls -ld /mnt/mydata  
drwxr-xr-x. 2 root root 6 Oct 7 08:59 /mnt/mydata  
[ec2-user@ip-172-31-7-49 ~]$ ls /mnt  
mydata  
[ec2-user@ip-172-31-7-49 ~]$ sudo mount /dev/nvme1n1 /mnt/mydata  
[ec2-user@ip-172-31-7-49 ~]$ df -h /mnt/mydata  
Filesystem      Size  Used Avail Use% Mounted on  
/dev/nvme1n1    2.0G  24K  1.8G   1% /mnt/mydata  
[ec2-user@ip-172-31-7-49 ~]$ sudo touch /mnt/mydata/testfile.txt  
[ec2-user@ip-172-31-7-49 ~]$ ls /mnt/mydata  
lost+found  testfile.txt  
[ec2-user@ip-172-31-7-49 ~]$ ls /dev/nvme1n1  
/dev/nvme1n1  
[ec2-user@ip-172-31-7-49 ~]$ findmnt /mnt/mydata  
TARGET SOURCE FSTYPE OPTIONS  
/mnt/mydata /dev/nvme1n1 ext4 rw,relatime,seclabel  
[ec2-user@ip-172-31-7-49 ~]$ ls -l /mnt/mydata  
total 16  
drwx-----. 2 root root 16384 Oct 7 08:58 lost+found  
-rw-r--r--. 1 root root    0 Oct 7 09:04 testfile.txt  
[ec2-user@ip-172-31-7-49 ~]$ echo "Zomato Menu - Pizza, Burger, Biryani, Sandwich" > ~/zomato_menu.txt  
[ec2-user@ip-172-31-7-49 ~]$ cat ~/zomato_menu.txt  
Zomato Menu - Pizza, Burger, Biryani, Sandwich  
[ec2-user@ip-172-31-7-49 ~]$ sudo cp ~/zomato_menu.txt /mnt/mydata/  
[ec2-user@ip-172-31-7-49 ~]$ ls -l /mnt/mydata/  
total 20  
drwx-----. 2 root root 16384 Oct 7 08:58 lost+found  
-rw-r--r--. 1 root root    0 Oct 7 09:04 testfile.txt  
-rw-r--r--. 1 root root   47 Oct 7 09:22 zomato_menu.txt  
[ec2-user@ip-172-31-7-49 ~]$ cat /mnt/mydata/zomato_menu.txt  
Zomato Menu - Pizza, Burger, Biryani, Sandwich  
[ec2-user@ip-172-31-7-49 ~]$  
[ec2-user@ip-172-31-7-49 ~]$ |
```

**Screenshot 1:** Zomato menu file successfully copied to and verified from the mounted EBS volume.

### 1. Create the menu file on the EC2 instance.

```
echo "Zomato Menu - Pizza, Burger, Biryani, Sandwich" >  
~/zomato_menu.txt
```

### 2. Copy the file to the mounted EBS volume.

```
sudo cp ~/zomato_menu.txt /mnt/mydata/
```

### 3. Verify the file from the mounted EBS volume.

```
ls -l /mnt/mydata/  
cat /mnt/mydata/zomato_menu.txt
```

The zomato\_menu.txt file was successfully displayed from /mnt/mydata, confirming that it was copied to and is accessible from the mounted EBS volume.

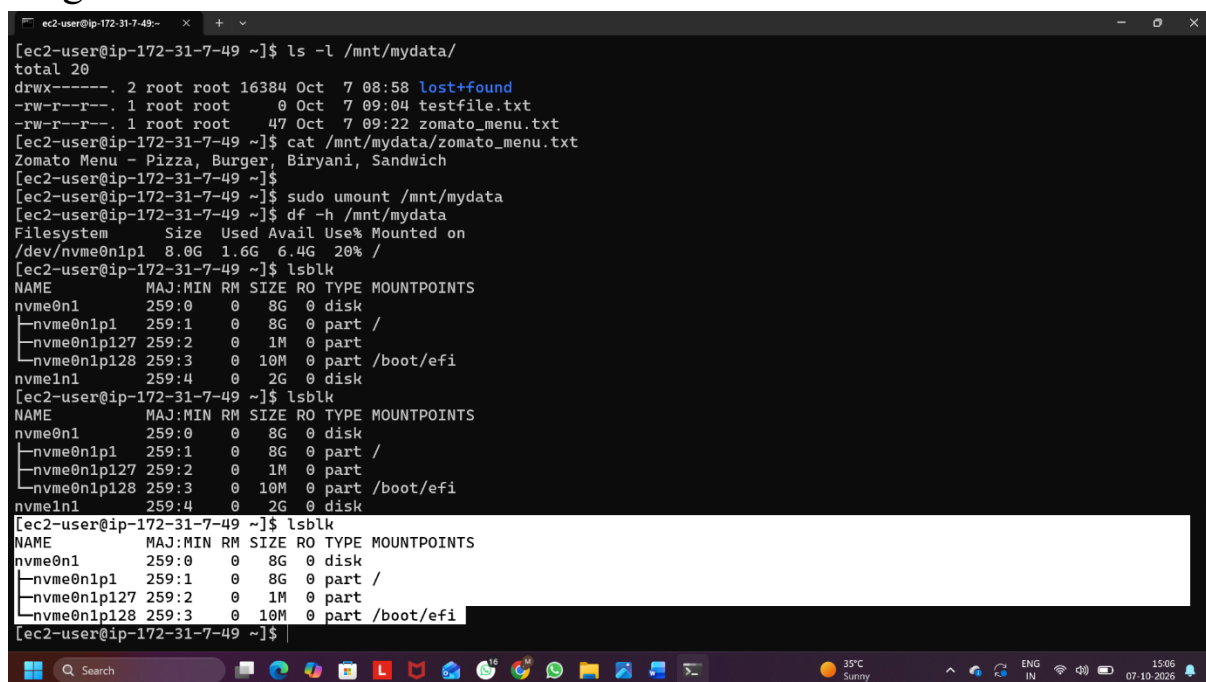
**Result:** The zomato\_menu.txt file was successfully created on the EC2 instance, copied to the mounted EBS volume, and verified from /mnt/mydata.

**Q5. Detach the EBS volume from your EC2 instance using the AWS Console, then reattach it to the same or a different instance. Describe what happens to the data you stored and why this behavior is useful for apps like Flipkart or Myntra order tracking.**

**Ans:**

## 1. Detach the EBS volume

The 2 GB EBS volume was detached from the EC2 instance using the AWS Management Console. After detachment, the volume was no longer visible as an attached disk on the EC2 instance.

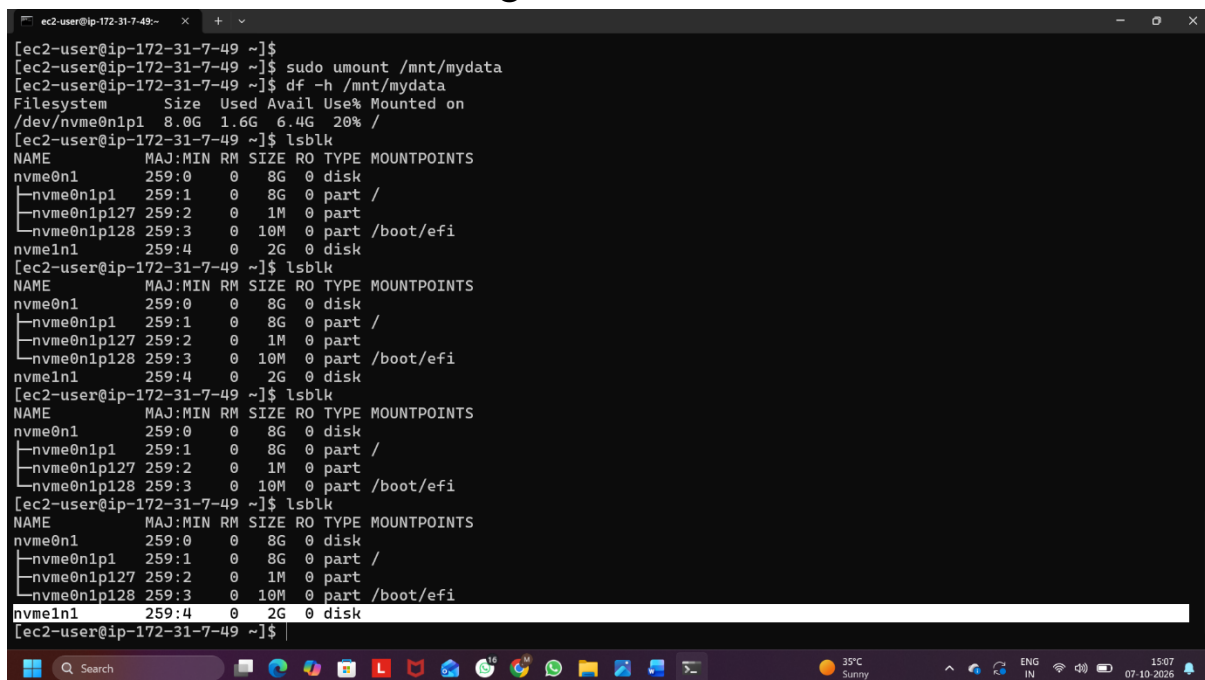


```
[ec2-user@ip-172-31-7-49 ~]$ ls -l /mnt/mydata/
total 20
drwx----- 2 root root 16384 Oct  7 08:58 lost+found
-rw-r--r-- 1 root root    0 Oct  7 09:04 testfile.txt
-rw-r--r-- 1 root root   47 Oct  7 09:22 zomato_menu.txt
[ec2-user@ip-172-31-7-49 ~]$ cat /mnt/mydata/zomato_menu.txt
Zomato Menu - Pizza, Burger, Biryani, Sandwich
[ec2-user@ip-172-31-7-49 ~]$ sudo umount /mnt/mydata
[ec2-user@ip-172-31-7-49 ~]$ df -h /mnt/mydata
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p1  8.0G  1.6G  6.4G  20% /
[ec2-user@ip-172-31-7-49 ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0    0    8G  0 disk 
└─nvme0n1p1 259:1    0    8G  0 part /
└─nvme0n1p127 259:2    0    1M  0 part 
└─nvme0n1p128 259:3    0   10M  0 part /boot/efi
nvme1n1     259:4    0    2G  0 disk 
[ec2-user@ip-172-31-7-49 ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0    0    8G  0 disk 
└─nvme0n1p1 259:1    0    8G  0 part /
└─nvme0n1p127 259:2    0    1M  0 part 
└─nvme0n1p128 259:3    0   10M  0 part /boot/efi
nvme1n1     259:4    0    2G  0 disk 
[ec2-user@ip-172-31-7-49 ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0    0    8G  0 disk 
└─nvme0n1p1 259:1    0    8G  0 part /
└─nvme0n1p127 259:2    0    1M  0 part 
└─nvme0n1p128 259:3    0   10M  0 part /boot/efi
[ec2-user@ip-172-31-7-49 ~]$
```

**Screenshot 1:** 2 GB EBS volume is no longer visible on the EC2 instance after detachment.

## 2. Reattach the EBS volume

The same EBS volume was reattached to the EC2 instance using **EC2 → Volumes → Actions → Attach volume**. After reattaching, the 2 GB volume became visible again on the EC2 instance.



```
ec2-user@ip-172-31-7-49:~$  
[ec2-user@ip-172-31-7-49 ~]$ sudo umount /mnt/mydata  
[ec2-user@ip-172-31-7-49 ~]$ df -h /mnt/mydata  
Filesystem      Size  Used Avail Use% Mounted on  
/dev/nvme0n1p1  8.0G  1.6G  6.4G  20% /  
[ec2-user@ip-172-31-7-49 ~]$ lsblk  
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS  
nvme0n1      259:0    0    8G  0 disk  
├─nvme0n1p1  259:1    0    8G  0 part /  
├─nvme0n1p127 259:2    0    1M  0 part  
├─nvme0n1p128 259:3    0   10M  0 part /boot/efi  
nvme1n1      259:4    0    2G  0 disk  
[ec2-user@ip-172-31-7-49 ~]$ lsblk  
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS  
nvme0n1      259:0    0    8G  0 disk  
├─nvme0n1p1  259:1    0    8G  0 part /  
├─nvme0n1p127 259:2    0    1M  0 part  
├─nvme0n1p128 259:3    0   10M  0 part /boot/efi  
nvme1n1      259:4    0    2G  0 disk  
[ec2-user@ip-172-31-7-49 ~]$ lsblk  
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS  
nvme0n1      259:0    0    8G  0 disk  
├─nvme0n1p1  259:1    0    8G  0 part /  
├─nvme0n1p127 259:2    0    1M  0 part  
├─nvme0n1p128 259:3    0   10M  0 part /boot/efi  
nvme1n1      259:4    0    2G  0 disk  
[ec2-user@ip-172-31-7-49 ~]$
```

**Screenshot 2:** Previously detached 2 GB EBS volume is successfully reattached to the EC2 instance.

### 3. Mount and verify the stored data

The reattached volume was mounted again at `/mnt/mydata`, and its contents were listed. The previously created files, including `zomato_menu.txt`, were available again.

```
ec2-user@ip-172-31-7-49:~$ lsblk
nvme0n1p1 259:1 0 8G 0 part /
nvme0n1p127 259:2 0 1M 0 part
nvme0n1p128 259:3 0 10M 0 part /boot/efi
nvme1n1 259:4 0 2G 0 disk
[ec2-user@ip-172-31-7-49 ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0 0   8G 0 disk
├─nvme0n1p1 259:1 0   8G 0 part /
├─nvme0n1p127 259:2 0   1M 0 part
├─nvme0n1p128 259:3 0  10M 0 part /boot/efi
└─nvme1n1   259:4 0   2G 0 disk
[ec2-user@ip-172-31-7-49 ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0 0   8G 0 disk
├─nvme0n1p1 259:1 0   8G 0 part /
├─nvme0n1p127 259:2 0   1M 0 part
├─nvme0n1p128 259:3 0  10M 0 part /boot/efi
└─nvme1n1   259:4 0   2G 0 disk
[ec2-user@ip-172-31-7-49 ~]$ sudo mount /dev/nvme1n1 /mnt/mydata
[ec2-user@ip-172-31-7-49 ~]$ ls -l /mnt/mydata
total 20
drwx-----. 2 root root 16384 Oct  7 08:58 lost+found
-rw-r--r--. 1 root root    0 Oct  7 09:04 testfile.txt
-rw-r--r--. 1 root root  47 Oct  7 09:22 zomato_menu.txt
[ec2-user@ip-172-31-7-49 ~]$ cat /mnt/mydata/zomato_menu.txt
Zomato Menu - Pizza, Burger, Biryani, Sandwich
[ec2-user@ip-172-31-7-49 ~]$
```

**Screenshot 3:** Previously stored files are accessible after remounting the reattached EBS volume.

## What happens to the data?

The data **remains stored on the EBS volume even after it is detached**. Detaching only disconnects the volume from the EC2 instance; it does not delete the data. When the same volume is reattached and mounted, the previously stored files become accessible again.

## Why is this useful for Flipkart/Myntra?

This persistence is useful for applications such as **Flipkart or Myntra** because important data such as **order details, transaction records, and application data** can remain safe even if an EC2 instance needs to be replaced or temporarily disconnected. The EBS volume can be reattached to another instance and the application can continue accessing the stored data.

